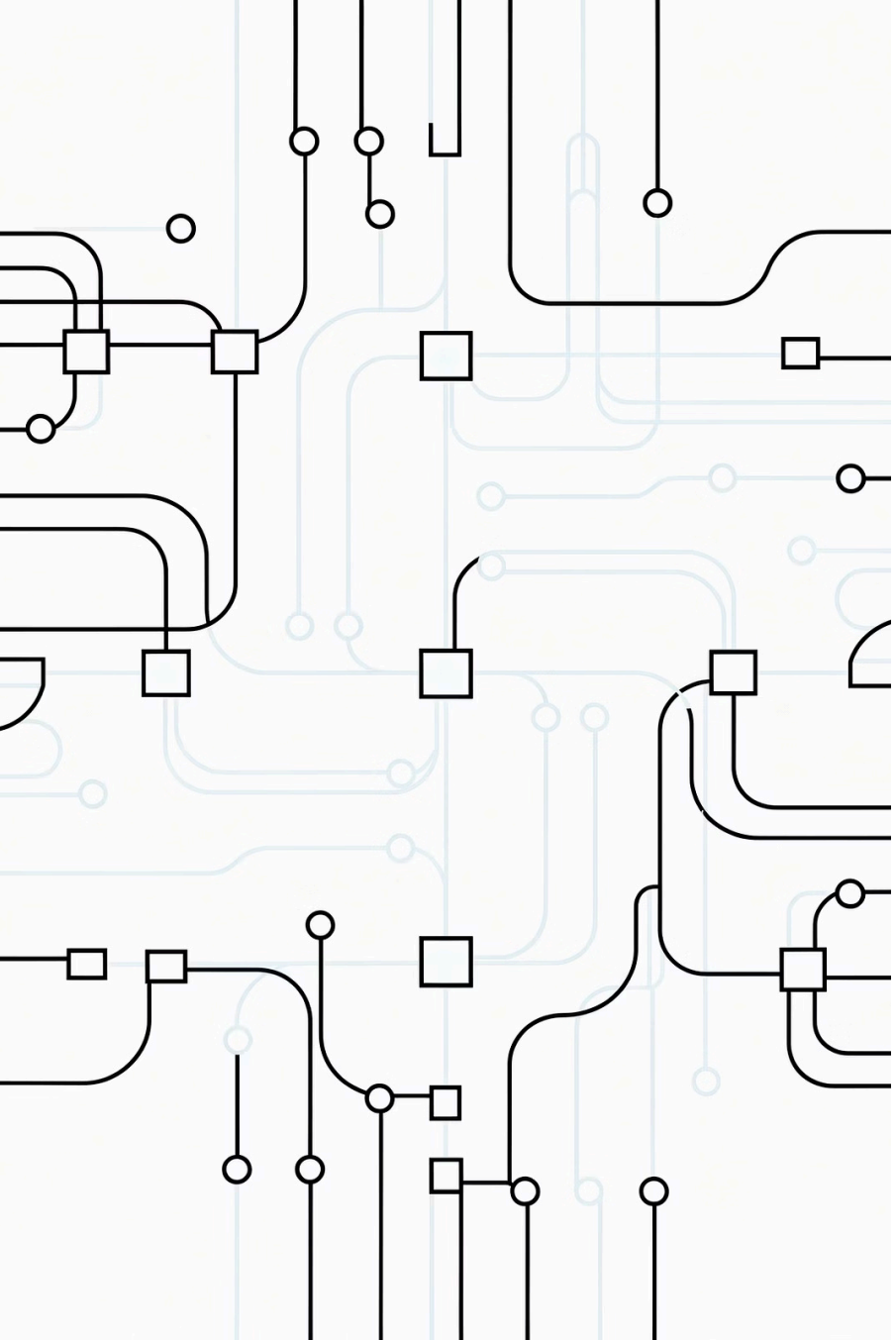


Semana 3: Resolviendo con decisiones y repeticiones en Java

Fundamentos de Programación | Experiencia 1

Docente: Jorge Carmona

¿Por qué controlar el flujo?

Hasta ahora, tus programas ejecutaban instrucciones **una tras otra**, de forma lineal. Pero los problemas reales requieren **decidir** y **repetir**. El control de flujo es la habilidad que transforma un programa básico en una solución inteligente.

→ **Programas secuenciales**

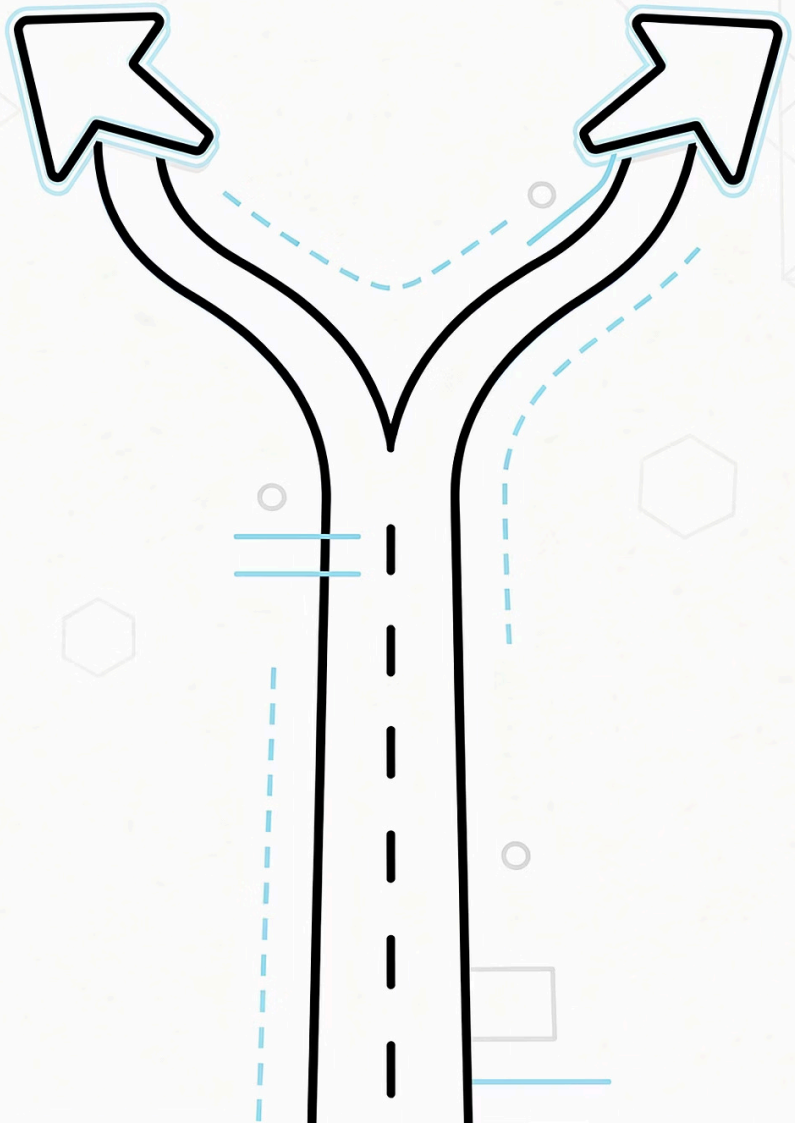
Ejecutan instrucciones de arriba hacia abajo, sin desviarse.

→ **Programas con decisiones**

Eligen qué camino tomar según una condición verdadera o falsa.

→ **Programas con repeticiones**

Repiten bloques de código de forma automática y controlada.



Objetivos de aprendizaje

SEMANA 3

1

Identificar estructuras condicionales

Reconocer cuándo y cómo usar `if-else` y `switch` para tomar decisiones en el código.

2

Aplicar bucles correctamente

Distinguir entre `for`, `while` y `do-while`, y elegir el más adecuado para cada situación.

3

Interpretar expresiones booleanas

Comprender y construir condiciones con operadores relacionales como `==`, `!=`, `>=`, entre otros.

Estructuras de selección: visión general

¿Qué son?

Son bloques de código que permiten al programa **tomar decisiones**. El programa evalúa una condición y ejecuta un camino u otro dependiendo del resultado.

- Condición **verdadera** → ejecuta un bloque
- Condición **falsa** → ejecuta otro bloque

¿Para qué sirven?

Permiten que el programa **responda de forma diferente** ante distintas situaciones, haciéndolo más flexible y útil.

- Validar entradas del usuario
- Clasificar datos según criterios
- Controlar el acceso o los resultados

En Java, las estructuras de selección principales son: `if-else` y `switch`.

SELECCIÓN

Estructura `if-else`

Sintaxis básica

```
if (condicion) {
    // se ejecuta si es true
} else {
    // se ejecuta si es false
}
```

Caso práctico: ¿par o impar?

```
int numero = 7;
if (numero % 2 == 0) {
    System.out.println("Par");
} else {
    System.out.println("Impar");
}
```

Buenas prácticas

Usa **llaves** `{}` siempre, aunque el bloque tenga solo una línea.

Escribe condiciones **claras y directas**, evita dobles negaciones.

El operador `%` entrega el **resto de la división**: `7 % 2 → 1` (impar).

SELECCIÓN

Estructura switch

¿Cuándo conviene usarlo?

Cuando la variable puede tomar **múltiples valores específicos** y cada uno requiere una acción distinta. Es más limpio que varios `if-else if` encadenados.

Caso práctico: día de la semana

```
int dia = 3;
switch (dia) {
    case 1: System.out.println("Lunes"); break;
    case 2: System.out.println("Martes"); break;
    case 3: System.out.println("Miércoles"); break;
    default: System.out.println("Día inválido");
}
```

Palabras clave del switch

case

Define un valor posible a comparar.

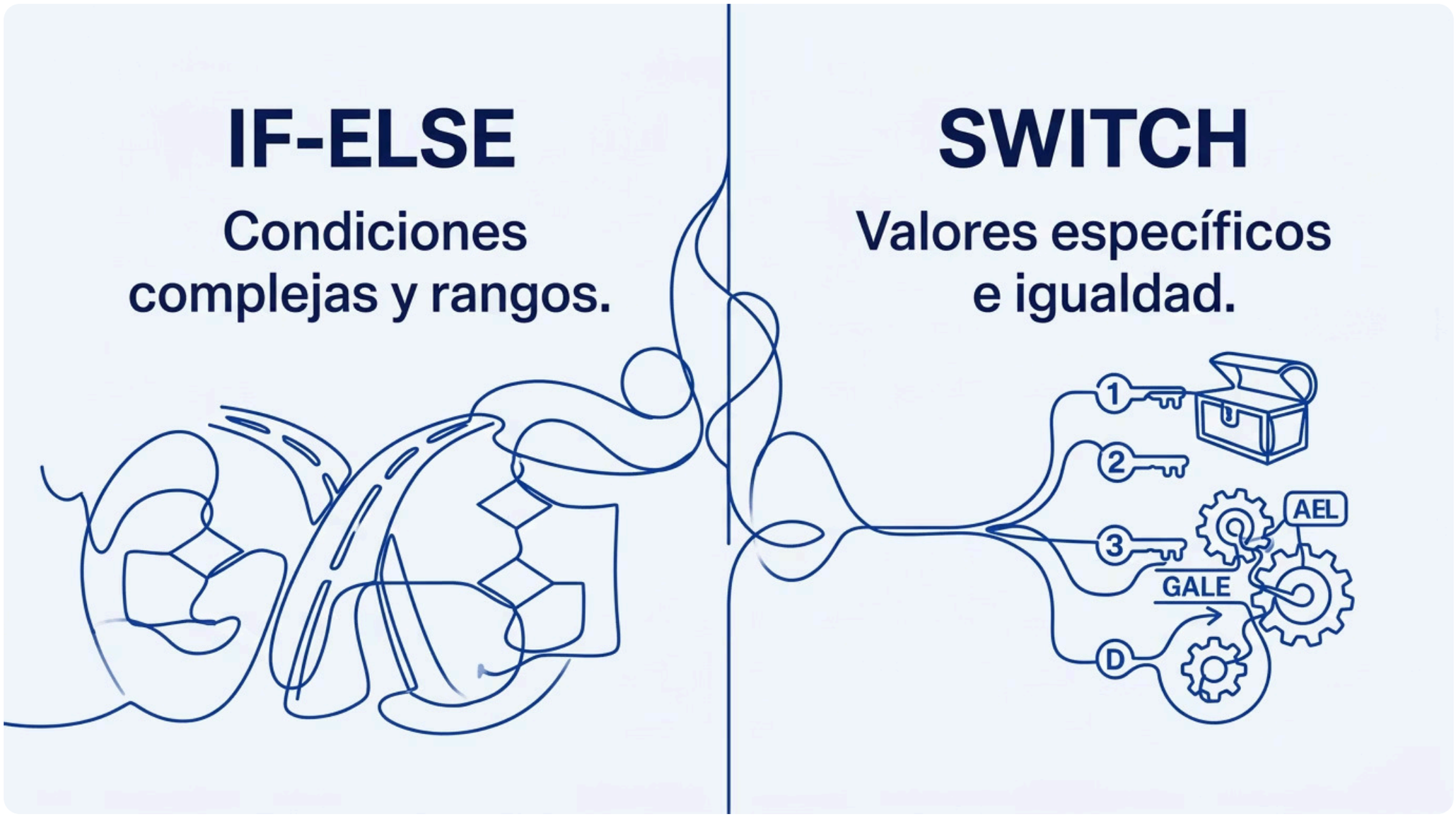
break

Detiene la ejecución del switch. Sin él, continúa al siguiente case.

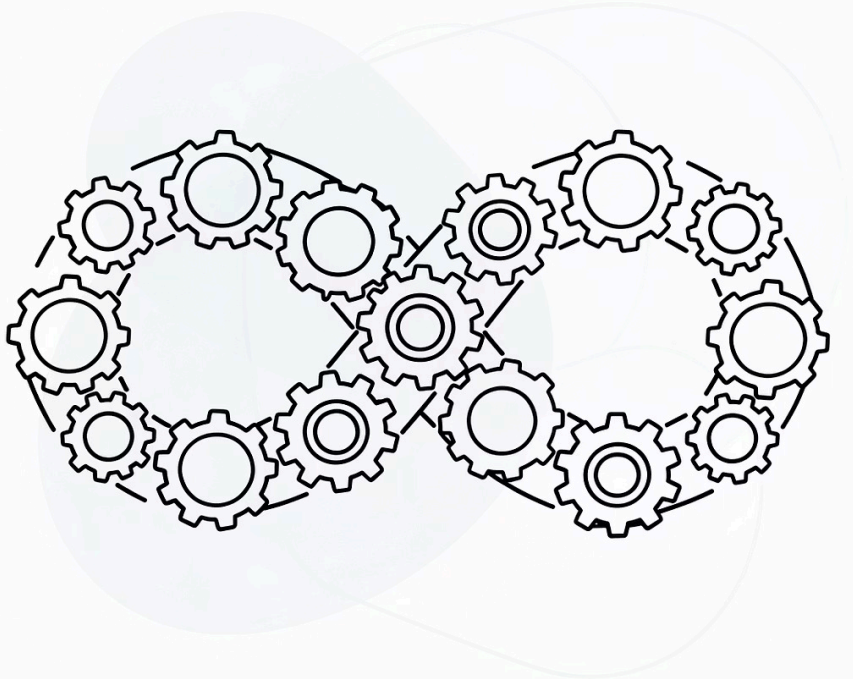
default

Se ejecuta cuando ningún case coincide. Es opcional pero recomendado.

Comparativa: `if-else` vs `switch`



Elige `if-else` cuando debas evaluar **rangos o condiciones complejas**; usa `switch` cuando compares **una variable contra valores exactos** y quieras un código más ordenado y legible.



Estructuras de iteración: visión general

Un **bucle** (o ciclo) es una estructura que permite **repetir un bloque de código** varias veces sin tener que escribirlo manualmente cada vez. Son fundamentales para automatizar tareas repetitivas.

Evitar repetición de código

En vez de escribir 100 líneas iguales, un bucle lo hace en 3.

Automatizar tareas

Procesar listas, calcular totales, validar entradas: todo con bucles.

Controlar la ejecución

El bucle se repite mientras se cumpla una condición definida por el programador.

ITERACIÓN

Bucle for

¿Cuándo usarlo?

Cuando sabes de antemano **cuántas veces** se repetirá el ciclo. Es el bucle más compacto porque define inicio, condición y avance en una sola línea.

Sus tres partes

```
for (inicio; condicion; actualizacion) {
    // bloque que se repite
}
```

Caso práctico: contar del 1 al 5

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
// Imprime: 1 2 3 4 5
```

Las 3 partes del for

1. Inicio

`int i = 1`

Se ejecuta una sola vez al comenzar.

2. Condición

`i <= 5`

Se evalúa antes de cada repetición.

3. Actualización

`i++`

Se ejecuta al final de cada vuelta.

ITERACIÓN

Bucles while y do-while

Bucle while

Repite el bloque **mientras la condición sea verdadera**. Si la condición es falsa desde el inicio, no ejecuta nada.

```
// Sumar hasta ingresar 0
int suma = 0, n = 1;
while (n != 0) {
    suma += n;
    n = scanner.nextInt();
}
System.out.println("Suma: " + suma);
```

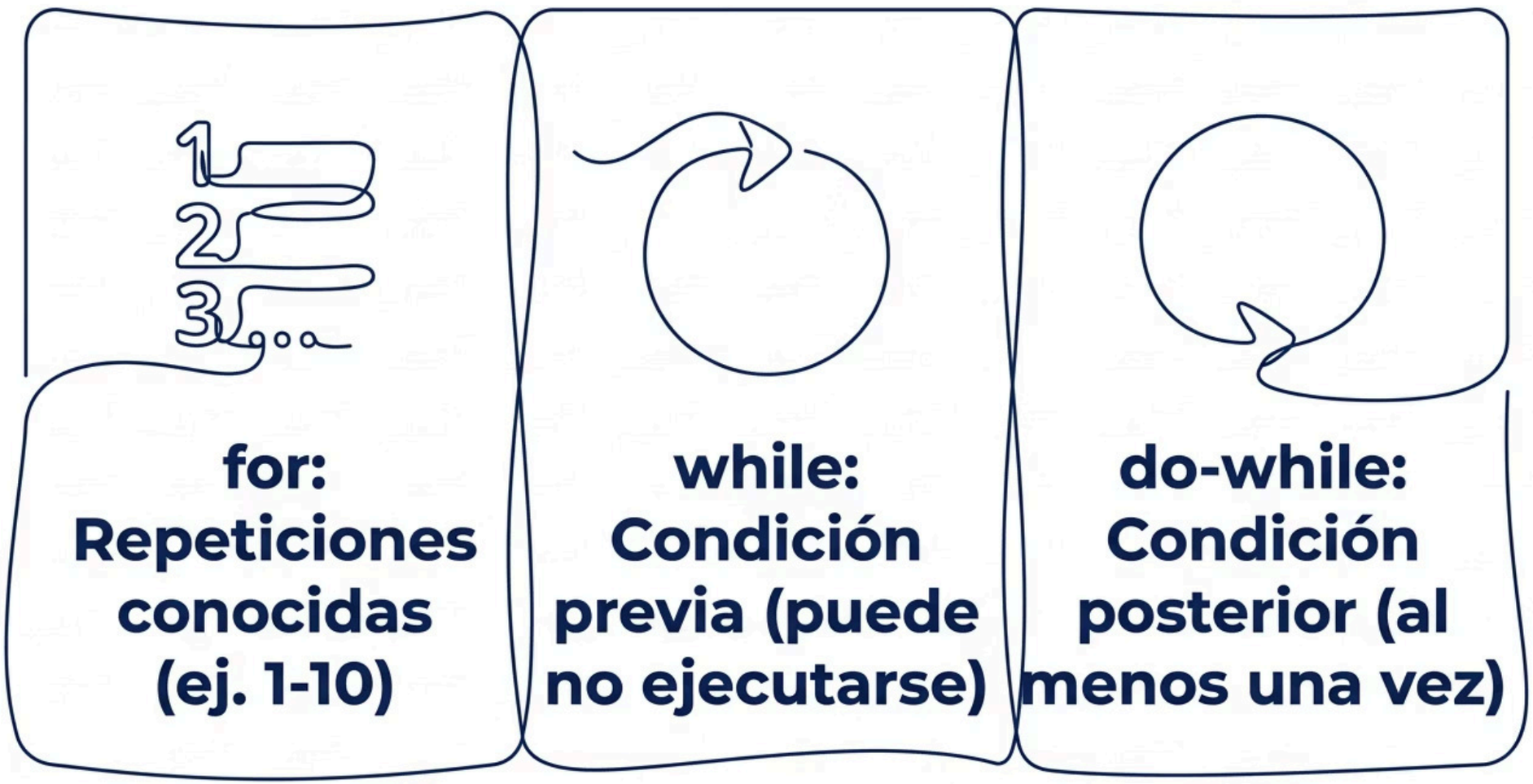
Bucle do-while

Ejecuta el bloque **al menos una vez** y luego verifica la condición. Ideal para validar datos ingresados por el usuario.

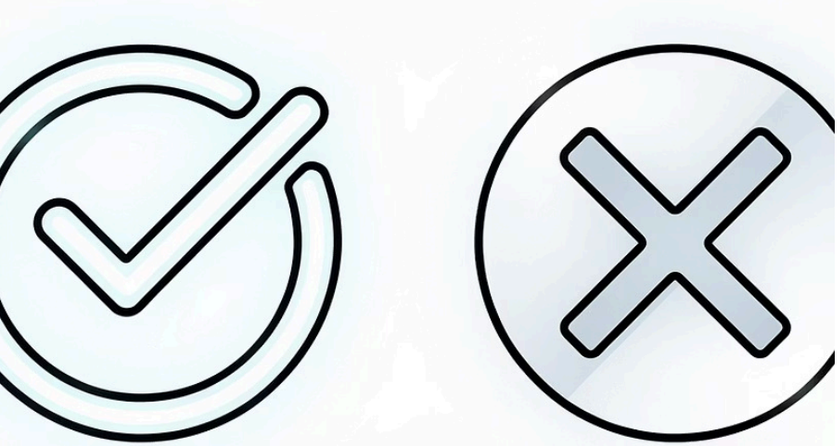
```
// Validar número positivo
int num;
do {
    System.out.print("Ingresa un número positivo: ");
    num = scanner.nextInt();
} while (num <= 0);
System.out.println("Válido: " + num);
```

Regla clave: Usa `while` cuando no sabes cuántas veces repetirás. Usa `do-while` cuando necesitas ejecutar el bloque al menos una vez antes de validar.

Comparativa de bucles: `for` vs `while` vs `do-while`



La clave está en preguntarte: **¿Sé cuántas veces se repetirá?** → usa `for`. **¿Depende de una condición externa?** → usa `while`. **¿Necesito ejecutarlo al menos una vez?** → usa `do-while`.



Expresiones booleanas simples

Una expresión booleana es una condición que solo puede resultar en **dos valores**: `true` (verdadero) o `false` (falso). Son la base de todas las estructuras de selección e iteración.

`==` Igual a

`edad == 18`

`!=` Distinto de

`saldo != 0`

`<` / `>` Menor / Mayor

`nota < 4.0`

`<=` / `>=` Menor/Mayor o igual

`edad >= 18`

⚠️ Cuidado: `==` compara valores (¿son iguales?), mientras que `=` asigna un valor. ¡No los confundas!

Actividades de práctica

REVISEMOS LO APRENDIDO

1

Aprobado o Reprobado

Escribe un programa con `if-else` que lea una nota y muestre **"Aprobado"** si es ⩾ 40 o **"Reprobado"** si es menor.

2

Saludo por turno

Usa `switch` con valores 1, 2, 3 para imprimir **"Buenos días"**, **"Buenas tardes"**, **"Buenas noches"**, y un `default` para turno inválido.

3

Conteo ascendente

Con un bucle `for`, imprime los números del 1 al 10 en una sola línea separados por espacios.

4

Simulación de despertador

Usa un `while` que imprima **"¡Despierta!"** exactamente 3 veces usando un contador.

Cierre: lo que aprendiste esta semana

01

Estructuras de selección

`if-else` para condiciones flexibles; `switch` para valores específicos.

02

Estructuras de iteración

`for` para repeticiones conocidas; `while` y `do-while` para condiciones dinámicas.

03

Expresiones booleanas

La base de toda decisión: condiciones que resultan en `true` o `false`.

> *"Controlar el flujo convierte un programa lineal en una solución inteligente y adaptable."*

💬 **Pregunta de reflexión:** ¿En qué situación real usarías un `if-else` y en cuál preferirías un `while`? Comparte tu respuesta con el curso.